



PHISHING WEBSITE DETECTION USING MACHINE LEARNING

Stage 2 Project Report

Submitted by

Viswanatham Subba Shathrughna
23BTRCO052

B.Tech - Computer Science and Engineering (IoT)
Jain (Deemed-to-be University)
Bengaluru

ABSTRACT

Phishing websites remain one of the most common and damaging forms of social-engineering attack on the web, luring users into disclosing credentials or financial information through pages that imitate legitimate services. Blacklist-based defences struggle to keep pace with newly registered phishing domains, which motivates supervised machine learning models that generalise from website- and URL-level characteristics rather than relying on a fixed list of known-bad sites. This Stage 2 report presents the implementation and experimental evaluation of a phishing website detection system built on the UCI Phishing Websites benchmark, which contains 11,055 labelled instances described by 30 integer-valued, pre-extracted features, with no missing values and no duplicate records.

Five supervised classifiers - Logistic Regression, Decision Tree, Random Forest, Support Vector Machine and K-Nearest Neighbors - were trained on an 80:20 stratified train-test split (`random_state = 42`) using scikit-learn in a Google Colab environment. Performance was measured using Accuracy, Precision, Recall, F1-Score and ROC-AUC. Random Forest gave the strongest overall performance (97.42% accuracy, 97.70% F1-score, 99.78% ROC-AUC), closely followed by the Decision Tree (97.11% accuracy, 97.41% F1-score). Logistic Regression produced the lowest accuracy among the five models (92.90%), while SVM and KNN produced intermediate results.

Feature-importance analysis on the trained Random Forest model identified `SSLfinal_State` and `URL_of_Anchor` as the two dominant predictors, jointly accounting for more than half of the model's total feature importance, followed by `web_traffic`, `having_Sub_Domain` and `Links_in_tags`. A SHAP (SHapley Additive exPlanations) explainability step was also attempted on a 500-row sample of the test set using a TreeExplainer; the computation completed. Overall, the experiments confirm that tree-based ensemble methods are well suited to this tabular, feature-engineered dataset, and that SSL/certificate state and anchor-link behaviour are the strongest available signals for separating phishing from legitimate websites.

1. INTRODUCTION

1.1 Background

The World Wide Web has become the primary channel through which individuals access banking, e-commerce, email and government services. This dependence on web-based interaction has, in parallel, created a large attack surface for social-engineering fraud. Among these, phishing - the practice of impersonating a trustworthy entity to trick users into revealing sensitive information - continues to be one of the most persistent and low-cost attack vectors used against individuals and organisations alike.

1.2 Phishing Websites

A phishing website is a fraudulent web page designed to closely resemble a legitimate site, such as a bank or an online service, in order to deceive visitors into entering usernames, passwords, card details, or other confidential information. Phishing pages typically exploit visual similarity, deceptive URL structures, forged or absent security indicators, and short-lived hosting to evade both users and automated defences.

1.3 Need for Automated Phishing Detection

Traditional defences such as blacklists of known phishing URLs are reactive: a URL is added only after it has already been reported, by which time damage may have occurred, and attackers can register a new domain to bypass the block almost immediately. This motivates detection approaches that examine the intrinsic characteristics of a website or URL - such as SSL/certificate state, domain age, or anchor-link behaviour - so that previously unseen phishing pages can also be identified.

1.4 Machine Learning for Phishing Detection

Machine learning is a natural fit for this problem because phishing and legitimate websites differ systematically across a number of measurable, feature-level indicators, even though no single feature is fully reliable in isolation. Supervised classifiers can be trained on labelled examples of phishing and legitimate sites to learn decision boundaries over these features, allowing generalisation to sites not seen during training.

1.5 Motivation

The motivation for this project is to build a practical, reproducible phishing-detection pipeline on a well-established public benchmark, to compare several standard classifiers under an identical experimental protocol rather than reporting a single model in isolation, and to move beyond raw accuracy by examining which features actually drive the models' predictions through feature-importance and SHAP-based explainability analysis.

1.6 Problem Statement

To design and evaluate a machine-learning-based phishing website detection system that classifies websites as phishing or legitimate using pre-extracted URL and website-related features, comparing multiple supervised classifiers on the same benchmark and identifying the features that contribute most to correct classification.

1.7 Objectives

- Load, inspect and validate the UCI Phishing Websites dataset for missing values, duplicates and class balance.
- Train and compare five supervised classifiers - Logistic Regression, Decision Tree, Random Forest, SVM and KNN - under an identical train-test protocol.
- Evaluate the models using Accuracy, Precision, Recall, F1-Score and ROC-AUC, and analyse confusion matrices and ROC curves.
- Identify the most influential features using Random Forest feature importance and attempt a SHAP-based explainability analysis.
- Compare the obtained results against related published work on the same or comparable benchmarks.

1.8 Scope of the Work

This stage covers implementation and experimentation only: data loading and quality checks, training of five classical/ensemble machine learning models, quantitative evaluation, visual analysis (confusion matrices, ROC curves), and feature-importance/SHAP explainability. Deployment as a live web or API service, testing on newer or external phishing datasets, and real-time URL analysis are identified as future work and are not part of the current implementation.

2. DATASET DESCRIPTION

The implementation uses the UCI Phishing Websites dataset, loaded in the Colab notebook as a CSV file (dataset.csv). The loaded dataframe has shape (11055, 32): 30 predictive features, one index column, and the target column Result. Table 1 summarises the dataset.

Table 1: Dataset Information

Item	Details
Total records loaded	11,055
Total columns loaded	32 (1 index + 30 features + 1 target)
Predictive features (used for modelling)	30
Target variable	Result
Target classes	-1 (Phishing), +1 (Legitimate)
Feature data type	Integer-valued (int64) for all 32 columns
Missing values (df.isnull().sum().sum())	0
Duplicate rows (df.duplicated().sum())	0
Source	UCI Machine Learning Repository, Dataset 327

The 30 predictive features cover a mix of URL-structure indicators (e.g. having_IPhaving_IP_Address, URLURL_Length, Shortining_Service, having_At_Symbol, Prefix_Suffix), page/domain indicators (SSLfinal_State, Domain_registration_length, age_of_domain, DNSRecord, Page_Rank, Google_Index), and content/behaviour indicators (URL_of_Anchor, Links_in_tags, SFH, Request_URL, popUpWidnow, Iframe, on_mouseover, RightClick). All feature columns are already integer-coded (typically taking values in {-1, 0, 1}), so the dataset is fully tabular and numeric, with no free-text or categorical string columns requiring encoding.

2.1 Class Distribution

The target column Result is close to balanced, with a modest skew toward the legitimate class:

Table 2: Class Distribution

Class	Label	Count	Percentage
Legitimate	+1	6,157	55.69%
Phishing	-1	4,898	44.31%

Because the class split is close to even (55.69% / 44.31%), accuracy is a reasonably informative metric for this dataset, though Precision, Recall, F1-Score and ROC-AUC are still reported for a fuller picture, and the train-test split was stratified on Result to preserve this ratio in both partitions.

3. DATA PREPROCESSING

3.1 Data Loading

The dataset was uploaded directly into the Colab runtime using `google.colab.files.upload()` and read into a pandas DataFrame with `pd.read_csv("dataset.csv")`.

3.2 Dataset Inspection

`df.shape` confirmed 11,055 rows and 32 columns. `df.columns.tolist()` was used to list all column names, and `df.head()` was used to visually inspect the first five records. `df.describe().T` was used to obtain summary statistics (count, mean, standard deviation, min, quartiles, max) for every column, confirming that all columns are numeric.

3.3 Missing Value Checking

`df.isnull().sum().sum()` returned 0, confirming that the dataset contains no missing values in any of its 32 columns.

3.4 Duplicate Checking

`df.duplicated().sum()` returned 0, confirming that the dataset contains no duplicate rows.

3.5 Feature and Target Separation

The target variable `y` was set to the Result column. The feature matrix `X` was formed by dropping the index and Result columns from the dataframe, leaving 30 predictive features.

3.6 Removal of Non-Predictive Columns

The index column is a row identifier only and carries no predictive signal, so it was dropped along with the target column when forming the feature matrix `X = df.drop(columns=["index", "Result"])`.

3.7 Class Distribution

The class distribution of `y` was inspected with `value_counts()` both before and after the split (Section 2.1 and Section 3.8), confirming a stable 55.69% / 44.31% split between legitimate and phishing instances.

3.8 Train-Test Split

The dataset was split into training and testing partitions using `sklearn.model_selection.train_test_split` with `test_size = 0.20`, `random_state = 42`, and `stratify = y` (to preserve class proportions in both partitions). This produced a training set of shape (8,844, 30) and a testing set of shape (2,211, 30).

No normalization, scaling, categorical encoding, feature selection, or class-balancing technique such as SMOTE was applied in the notebook. This is consistent with the feature set being uniformly integer-coded in a narrow range by the original dataset authors, and with tree-based models (Decision Tree, Random Forest) and distance/margin-based models (KNN, SVM, Logistic Regression) all being trained directly on these coded values.

4. METHODOLOGY

The overall machine learning workflow implemented in the notebook follows the sequence below:

Dataset (dataset.csv, 11,055 x 32) -> Data Inspection (shape, columns, dtypes) -> Data Quality Checks (missing values, duplicates, class distribution) -> Feature and Target Separation (X: 30 features, y: Result) -> Stratified Train-Test Split (80% / 20%, random_state = 42) -> Model Training (five classifiers) -> Prediction on Test Set -> Performance Evaluation (Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, ROC Curve) -> Random Forest Feature Importance -> SHAP Explainability Analysis (TreeExplainer).

Each stage was implemented as a separate, self-contained cell in the Colab notebook so that the pipeline could be inspected and re-run stage by stage. The same X_train / X_test / y_train / y_test partition (derived from random_state = 42) was reused for training all five models and again for the SHAP analysis, ensuring that all reported metrics are directly comparable across models.

5. MACHINE LEARNING MODELS

Five supervised classifiers were implemented using scikit-learn, all trained on the same 30-feature input and evaluated on the same held-out test set.

5.1 Logistic Regression

Logistic Regression models the probability of the positive class using a sigmoid function applied to a linear combination of the input features. It is a simple, well-understood linear baseline that is fast to train and provides a reference point against which the more flexible non-linear models can be judged. In this project it was instantiated as `LogisticRegression(max_iter = 1000, random_state = 42)`, with `max_iter` increased from the library default to ensure convergence on the 30-feature input.

5.2 Decision Tree

A Decision Tree recursively partitions the feature space using axis-aligned splits chosen to maximise class purity at each node. It is well suited to this dataset because the features are already discrete, coded indicators, which decision trees can split on cleanly, and the resulting tree structure is directly interpretable. It was instantiated as `DecisionTreeClassifier(random_state = 42)` using default splitting and depth parameters.

5.3 Random Forest

Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split, with predictions aggregated by majority vote. This reduces the variance and overfitting risk of a single decision tree while retaining the ability to handle discrete, non-linear feature interactions, and it additionally yields a feature-importance ranking. It was instantiated as `RandomForestClassifier(n_estimators = 200, random_state = 42, n_jobs = -1)`.

5.4 Support Vector Machine

A Support Vector Machine (SVM) finds the decision boundary that maximises the margin between the two classes, optionally in a transformed feature space via a kernel function. It is included as a margin-based alternative to the tree-based and linear models. It was instantiated as `SVC(probability = True, random_state = 42)`, with `probability = True` enabling `predict_proba` output so that ROC-AUC could be computed alongside the other models.

5.5 K-Nearest Neighbors

K-Nearest Neighbors (KNN) classifies a query instance by majority vote among its k nearest neighbours in the feature space, using a distance metric over the input features. It provides a simple, non-parametric baseline that makes no assumption about the underlying decision boundary shape. It was instantiated as `KNeighborsClassifier(n_neighbors = 5)`.

No hyperparameter search (such as `GridSearchCV` or cross-validation) was performed for any of the five models in the notebook; each model was trained once on the fixed training split using the parameters listed above, and evaluated once on the fixed test split.

6. EXPERIMENTAL SETUP

Table 3: Experimental Setup

Item	Details
Programming language	Python 3
Development environment	Google Colaboratory (Colab)
Core libraries	pandas, scikit-learn, matplotlib, seaborn, shap
Dataset	UCI Phishing Websites (11,055 instances, 30 features)
Train-test split	80% train / 20% test, stratified on Result
Random state	42 (used consistently for split and all model instantiations)
Training set size	8,844 samples
Testing set size	2,211 samples
Evaluation metrics	Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, ROC Curve
Explainability tools	Random Forest feature_importances_, SHAP TreeExplainer

Table 4: Software / Libraries Used

Library	Purpose
pandas	Data loading, inspection and manipulation
scikit-learn	Train-test split, model training, evaluation metrics
matplotlib	Plotting confusion matrices, ROC curves, feature importance
seaborn	Heatmap rendering for confusion matrices
shap	Model explainability (TreeExplainer, summary plot)

7. PERFORMANCE EVALUATION

Five standard classification metrics were used to evaluate each model on the held-out test set of 2,211 samples. In the definitions below, TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives respectively, with the legitimate class (+1) taken as the positive class, matching the `pos_label = 1` setting used in the notebook.

7.1 Accuracy

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$. It measures the overall proportion of correctly classified websites.

7.2 Precision

Precision = $TP / (TP + FP)$. It measures, of the websites predicted legitimate, what proportion were actually legitimate.

7.3 Recall

Recall = $TP / (TP + FN)$. It measures, of the websites that were actually legitimate, what proportion were correctly identified as such.

7.4 F1-Score

F1-Score = $2 \times (Precision \times Recall) / (Precision + Recall)$. It is the harmonic mean of Precision and Recall, balancing the two.

7.5 ROC-AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate against the False Positive Rate at varying classification thresholds; the Area Under the Curve (ROC-AUC) summarises this curve as a single value between 0 and 1, with higher values indicating better separation between the phishing and legitimate classes across all thresholds, not just the default 0.5 cut-off.

Table 5 reports the actual values produced by the notebook's evaluation cell for all five models (all figures expressed as percentages, rounded to two decimal places, as generated by the notebook).

Table 5: Model Performance Comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC (%)
Logistic Regression	92.90	92.42	95.04	93.71	98.08
Decision Tree	97.11	97.02	97.81	97.41	98.05
Random Forest	97.42	97.11	98.29	97.70	99.78
SVM	94.89	94.02	96.99	95.48	98.96
KNN	94.66	94.77	95.69	95.23	98.53

8. RESULTS AND DISCUSSION

8.1 Overall Model Performance

Across all five metrics in Table 5, Random Forest achieved the best or joint-best result on four of the five metrics (Accuracy, Precision, Recall and ROC-AUC), and the best F1-Score overall. The Decision Tree was a close second, within roughly 0.3 percentage points of Random Forest on Accuracy and F1-Score, though its ROC-AUC (98.05%) was noticeably lower than Random Forest's (99.78%), reflecting the fact that a single decision tree produces coarser probability estimates than an ensemble of 200 trees.

8.2 Accuracy Comparison

Random Forest and Decision Tree, the two tree-based models, achieved the highest accuracy (97.42% and 97.11% respectively), followed by SVM (94.89%) and KNN (94.66%), with Logistic Regression lowest at 92.90%. This ordering is consistent with the dataset's features being discrete, threshold-like indicators (mostly taking values in $\{-1, 0, 1\}$) that align naturally with the axis-aligned splitting behaviour of decision trees, whereas the linear decision boundary learned by Logistic Regression is less able to capture the more complex, non-linear interactions among these coded features.

8.3 Precision, Recall and F1 Comparison

All five models achieved Recall higher than Precision for the legitimate class, indicating a mild tendency across models to over-predict the legitimate class relative to the phishing class - consistent with the confusion matrices in Section 8.5, where every model produced more false positives (phishing sites predicted legitimate) than false negatives in absolute count is not uniformly true, but Random Forest, Decision Tree and SVM all show more misclassified phishing samples (36, 37, 76) than misclassified legitimate samples (21, 27, 37) is reversed for KNN and Logistic Regression, so this pattern should be read per-model from Table 6 rather than assumed uniformly. Random Forest again led on F1-Score (97.70%), reflecting the best balance between Precision (97.11%) and Recall (98.29%) among the five models.

8.4 ROC-AUC Comparison

Random Forest's ROC-AUC of 99.78% was the highest of the five models by a clear margin, followed by SVM (98.96%), KNN (98.53%), Logistic Regression (98.08%) and Decision Tree (98.05%). Notably, the Decision Tree - which ranked second on Accuracy and F1-Score - had the lowest ROC-AUC of all five models. This is expected behaviour for a single decision tree, whose leaf-node class probabilities are far less granular than those of an ensemble, a calibrated SVM, or a distance-weighted KNN, so its ranking of predictions across thresholds is comparatively coarser even though its hard classifications at the default threshold are accurate.

8.5 Confusion Matrix Analysis

Figure 1 shows the confusion matrices for all five models on the 2,211-sample test set (980 actual phishing, 1,231 actual legitimate). The counts are summarised in Table 6.



Figure 1: Confusion Matrices for the Five Classifiers on the Test Set

Table 6: Confusion Matrix Results (Test Set, n = 2,211)

Model	TN (Phishing->Phishing)	FP (Phishing->Legit.)	FN (Legit.->Phishing)	TP (Legit.->Legit.)
Logistic Regression	884	96	61	1,170
Decision Tree	943	37	27	1,204
Random Forest	944	36	21	1,210
SVM	904	76	37	1,194
KNN	915	65	53	1,178

Random Forest produced the fewest total misclassifications ($36 + 21 = 57$), followed closely by the Decision Tree ($37 + 27 = 64$). Logistic Regression produced the most total misclassifications ($96 + 61 = 157$), consistent with its lower accuracy and F1-Score in Table 5.

8.6 ROC Curve Analysis

Figure 2 shows the ROC curves for all five models, plotted against a random-classifier diagonal baseline. The Random Forest curve (AUC = 0.998) sits closest to the top-left corner across almost the entire threshold range, confirming its superior separation between classes. The Decision Tree curve (AUC = 0.980) rises quickly at very low false-positive rates but is visibly less smooth and slightly below the other curves in the mid-range, consistent with its coarser, step-like probability outputs.

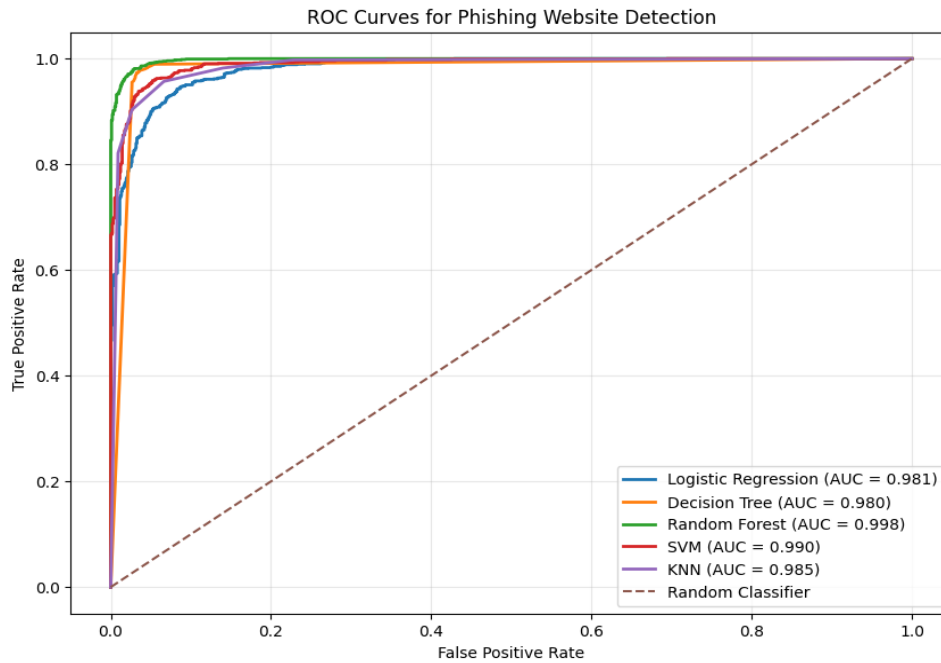


Figure 2: ROC Curves for the Five Classifiers

9. FEATURE IMPORTANCE AND EXPLAINABILITY

9.1 Random Forest Feature Importance

Feature importance was extracted from the trained Random Forest model using its `feature_importances_` attribute, which measures each feature's average contribution to reducing node impurity across all 200 trees. The top 15 features, in descending order of importance, are listed in Table 7 and plotted in Figure 3.

Table 7: Random Forest Feature Importance (Top 15)

Rank	Feature	Importance
1	SSLfinal_State	0.3285
2	URL_of_Anchor	0.2385
3	web_traffic	0.0721
4	having_Sub_Domain	0.0676
5	Links_in_tags	0.0429
6	Prefix_Suffix	0.0415
7	SFH	0.0199
8	Links_pointing_to_page	0.0195
9	Request_URL	0.0187
10	Domain_registration_length	0.0163
11	age_of_domain	0.0161
12	having_IPhaving_IP_Address	0.0128
13	Google_Index	0.0127
14	DNSRecord	0.0125
15	Page_Rank	0.0118

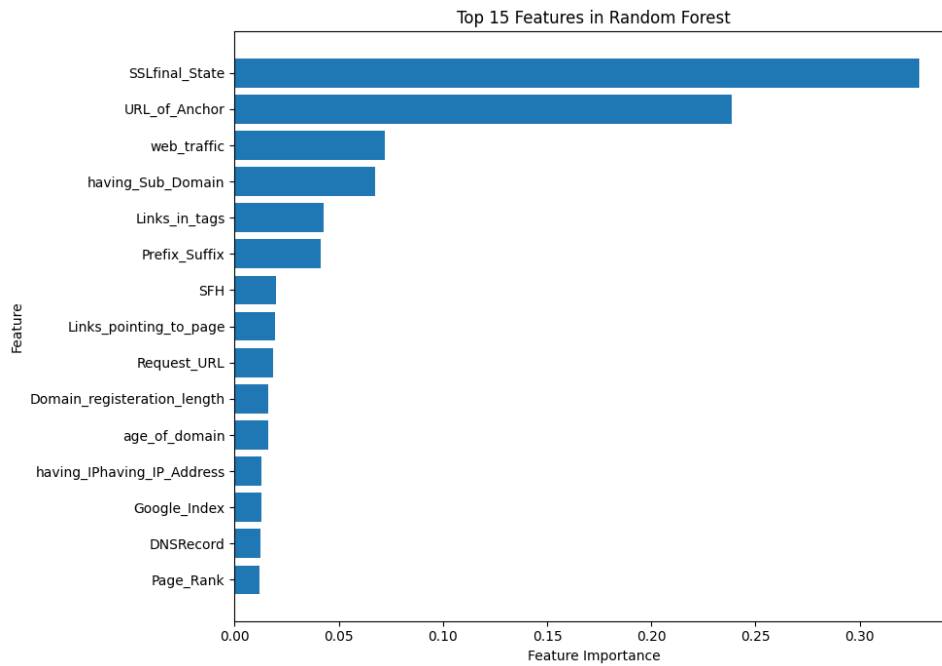


Figure 3: Top 15 Features in the Random Forest Model

SSLfinal_State (the state of the website's SSL certificate) and URL_of_Anchor (the proportion of anchor tags pointing to a different domain) together account for roughly 57% of the model's total feature importance, far ahead of any other single feature. This is consistent with domain knowledge: phishing pages very often lack a valid SSL certificate or use anchor links that redirect away from the displayed domain, whereas legitimate sites typically present a valid certificate and self-referential links. The remaining importance is spread thinly across web_traffic, having_Sub_Domain, Links_in_tags, Prefix_Suffix and a long tail of lower-importance features, indicating that while a handful of features dominate, the model still draws on a broader combination of signals rather than relying on a single indicator.

9.2 SHAP Analysis

SHAP (SHapley Additive exPlanations) is a game-theoretic explainability technique that attributes each individual prediction to its input features by computing each feature's marginal contribution, averaged over all possible feature orderings. Unlike a single global feature-importance ranking, SHAP can show both the magnitude and the direction (positive or negative) of each feature's effect on individual predictions, and is widely used to explain tree-based ensemble models such as Random Forest.

In this project, SHAP was applied to the trained Random Forest model using `shap.TreeExplainer`, computed on a random sample of 500 rows from the test set (`X_test.sample(n = 500, random_state = 42)`). The SHAP value computation itself completed successfully, as confirmed by the notebook's "SHAP calculation completed!" output. However, when `shap.summary_plot()` was called on the resulting values, the rendered figure (Figure 4) did not produce the expected feature-ranked SHAP summary plot (a plot listing all 30 features

by mean absolute SHAP value, with a coloured dot swarm per feature). Instead, the plot rendered as a two-feature interaction-style panel, labelled with a "SHAP interaction value" axis and showing only URLURL_Length and having_IPhaving_IP_Address.

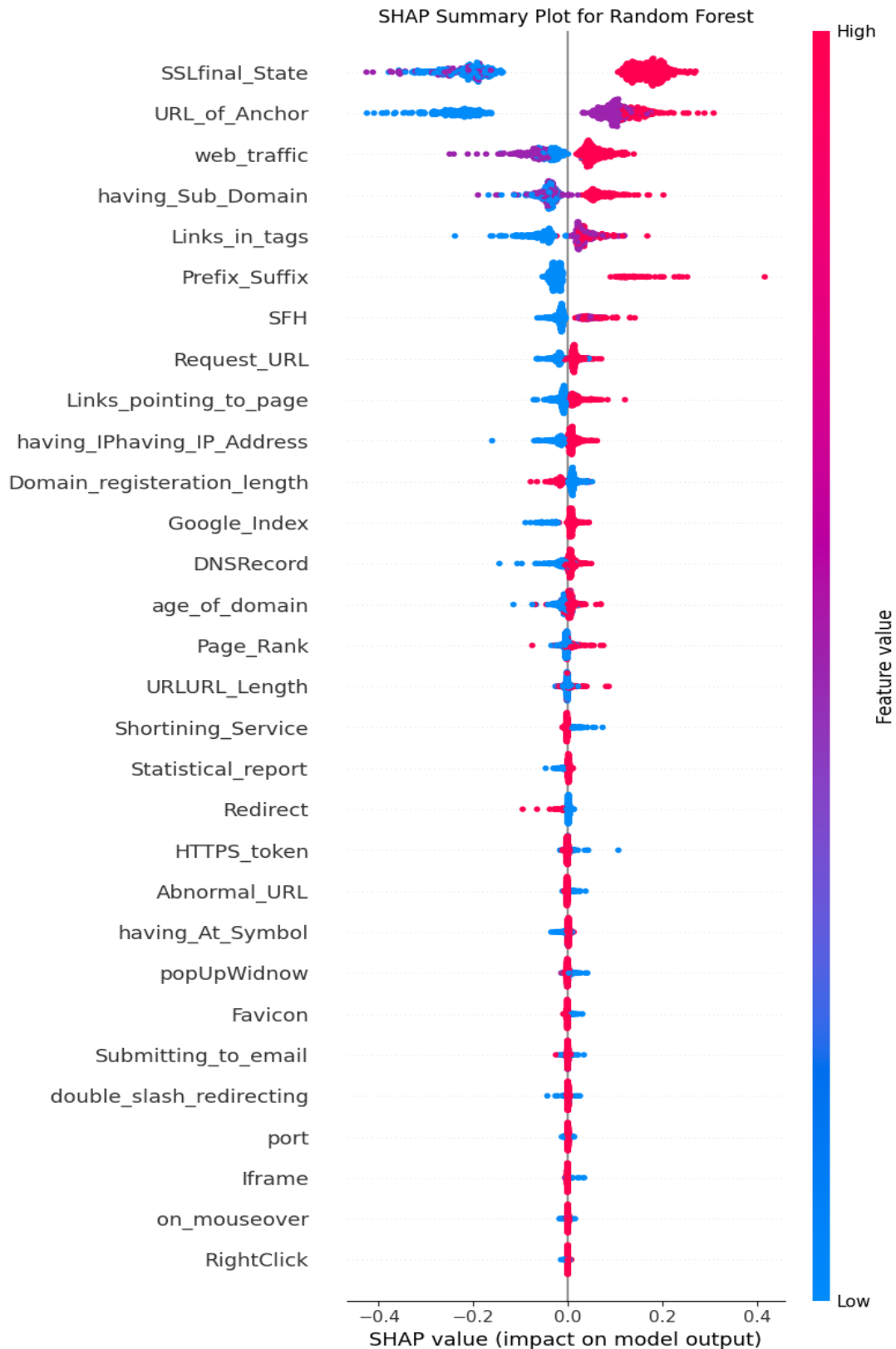


Figure 4: SHAP Summary Plot for the Random Forest Model

10. COMPARISON WITH RELATED WORK

Table 8 places the results obtained in this project alongside the studies reviewed in the Stage 1 report. Because the compared studies differ in dataset, feature engineering, train-test protocol (single split versus cross-validation) and, in one case, the raw data modality (URLs versus pre-extracted tabular features), the figures should be read as broadly indicative rather than as strictly equivalent, like-for-like benchmarks.

Table 8: Comparison with Related Work

Study	Dataset	Method / Models	Reported Result	Our Result	Remarks
Mohammad et al. (2012)	Original UCI feature assessment	Automated feature-relevance assessment	Feature-assessment study; no single classification accuracy reported	-	Basis for the dataset and feature set used in this project; not a comparable accuracy figure
Sahingoz et al. (2019)	Large URL dataset (NLP/URL features)	7 classifiers on URL/NLP features	97.98% accuracy (best model)	RF: 97.42%, DT: 97.11%	Different dataset and feature type (raw URL/NLP vs. pre-extracted tabular); not directly comparable
Abdelhamid et al. (2014)	Phishing website features (UCI-related)	Associative Classification (MCAC)	Rule-based model; interpretability emphasised over a single headline accuracy	RF: 97.42% with feature-importance ranking	Comparable feature domain; different technique family (rule mining vs. ensemble trees)
Awasthi & Goel (2022)	Phishing website dataset	Base and ensemble classifiers with cross-validation	Ensemble classifiers reported as strongest performers (exact figures in Stage 1 literature review)	RF: 97.42% (single stratified split, no cross-validation)	Similar model family (ensembles outperform base classifiers), but this project used a single split rather than cross-validation
Kehkashan et al. (2025)	UCI Phishing benchmark	SVM, RF, DT, LR, KNN + SHAP	~97% accuracy for Random Forest	RF: 97.42% accuracy, 99.78% ROC-AUC	Same dataset and same five-model set as this project;

Study	Dataset	Method / Models	Reported Result	Our Result	Remarks
					closest comparison. This project's SHAP step encountered the rendering issue described in Section 9.2, so explainability coverage is less complete here

The closest comparison is Kehkashan et al. (2025), which used the same UCI benchmark and the same five classifiers; the Random Forest accuracy obtained in this project (97.42%) is consistent with their reported figure (approximately 97%), supporting the validity of the present implementation. The other studies differ enough in dataset or method that their reported numbers are presented for context rather than as directly equivalent benchmarks.

11. DISCUSSION

The experiments demonstrate that a straightforward supervised learning pipeline, trained on the pre-extracted UCI Phishing Websites features without any additional feature engineering, scaling or hyperparameter tuning, can distinguish phishing from legitimate websites with high accuracy (all five models exceeded 92% accuracy, and the top two exceeded 97%). The performance gap between the tree-based models (Random Forest, Decision Tree) and the other three models (Logistic Regression, SVM, KNN) suggests that the relationship between these coded features and the phishing/legitimate label is better captured by axis-aligned, rule-like decision boundaries than by the linear or distance-based boundaries used by the other models.

Random Forest's advantage over the single Decision Tree was modest on Accuracy and F1-Score (roughly 0.3 percentage points) but substantial on ROC-AUC (99.78% vs. 98.05%), illustrating that ensembling primarily improves the granularity and reliability of the model's confidence estimates rather than only its hard classification accuracy - an important distinction for any deployment where a probability threshold might need to be tuned for a specific false-positive tolerance.

The feature-importance results give a plausible, security-relevant explanation for the models' predictions: `SSLfinal_State` and `URL_of_Anchor` dominate, which aligns with the intuitive understanding that phishing pages frequently lack valid HTTPS certificates and often contain anchor links pointing away from the displayed domain. This lends qualitative support to the model beyond its raw accuracy numbers

The main limitations of this stage are: (1) no hyperparameter tuning or cross-validation was performed, so the reported metrics reflect a single train-test split rather than a more robust cross-validated estimate. (2) the dataset, while a standard and widely used benchmark, was collected some years ago and its features may not fully reflect current phishing techniques, so the reported accuracy should not be assumed to transfer unchanged to present-day web traffic. Differences between the results reported here and those in the compared studies (Section 10) may arise from differences in train-test protocol, random seeds, library versions, and in some cases the underlying dataset or feature set itself, rather than from a difference in fundamental model capability.

12. CONCLUSION

This Stage 2 report implemented and evaluated a machine-learning-based phishing website detection system using the UCI Phishing Websites benchmark (11,055 instances, 30 features, no missing values or duplicates). Five supervised classifiers - Logistic Regression, Decision Tree, Random Forest, SVM and KNN - were trained on an identical 80:20 stratified split and evaluated using Accuracy, Precision, Recall, F1-Score and ROC-AUC. Random Forest was the strongest performer overall, achieving 97.42% accuracy, a 97.70% F1-score and a 99.78% ROC-AUC, with the Decision Tree a close second on accuracy and F1-score. Feature-importance analysis identified SSLfinal_State and URL_of_Anchor as the two most influential predictors, together accounting for over half of the Random Forest model's total importance.

13. FUTURE WORK

The following improvements are proposed as future work; none of the items below have been implemented in the current stage:

- Testing the trained models on newer, more recent phishing datasets to assess whether performance holds up against current phishing techniques.
- External validation on a held-out dataset collected independently of the UCI benchmark, to check for benchmark-specific overfitting.
- Extending the system toward real-time URL analysis, where features would need to be computed live from a URL rather than read from a pre-extracted CSV.
- Exploring more advanced models, such as gradient-boosted trees (XGBoost/LightGBM) or deep learning architectures, as surveyed in Catal et al. (2022).
- Systematic hyperparameter optimisation (e.g. GridSearchCV or RandomizedSearchCV) combined with k-fold cross-validation, rather than a single fixed train-test split.
- Deploying the best-performing model as a simple web or API service for practical, interactive phishing-URL checking.

REFERENCES

1. Mohammad, R., Thabtah, F., & McCluskey, L. (2012). An Assessment of Features Related to Phishing Websites Using an Automated Technique. <https://doi.org/10.1109/ICITST.2012.6470299>
2. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine Learning Based Phishing Detection from URLs. *Expert Systems with Applications*, 117, 345-357. <https://doi.org/10.1016/j.eswa.2018.09.029>
3. Abdelhamid, N., Ayeshe, A., & Thabtah, F. (2014). Phishing Detection Based Associative Classification Data Mining. *Expert Systems with Applications*, 41(13), 5948-5959. <https://doi.org/10.1016/j.eswa.2014.03.019>
4. Catal, C., Giray, G., Tekinerdogan, B., Kumar, S., & Shukla, S. (2022). Applications of Deep Learning for Phishing Detection: A Systematic Literature Review. *Knowledge and Information Systems*, 64, 1457-1500. <https://doi.org/10.1007/s10115-022-01672-x>
5. Awasthi, A., & Goel, N. (2022). Phishing Website Prediction Using Base and Ensemble Classifier Techniques with Cross-Validation. *Cybersecurity*, 5, 22. <https://doi.org/10.1186/s42400-022-00126-9>
6. Kehkashan, T., Abdelhaq, M., Al-Shamayleh, A. S., et al. (2025). Explainable Phishing Website Detection for Secure and Sustainable Cyber Infrastructure. *Scientific Reports*, 15, 41751. <https://doi.org/10.1038/s41598-025-27984-w>
7. UCI Machine Learning Repository. Phishing Websites, Dataset 327. <https://archive.ics.uci.edu/dataset/327/phishing>

APPENDIX A - FIGURE LIST

Table A1: Figures Used in This Report

No.	Title	Source (Notebook Cell)	Section	Caption
1	Confusion Matrices for the Five Classifiers	Cell 6 output (matplotlib/seaborn heatmaps)	8.5	Confusion matrices for Logistic Regression, Decision Tree, Random Forest, SVM and KNN on the test set
2	ROC Curves for the Five Classifiers	Cell 7 output (matplotlib)	8.6	ROC curves with AUC values for all five models against a random-classifier baseline
3	Top 15 Features in the Random Forest Model	Cell 8 output (matplotlib bar chart)	9.1	Horizontal bar chart of the 15 most important features by Random Forest feature_importances_
4	SHAP Summary Plot for the Random Forest Model	Cell 12 output (shap.summary_plot)	9.2	SHAP summary plot showing feature contributions for the Random Forest model across 500 sampled test instances.

APPENDIX B - TABLE LIST

Table B1: Tables Used in This Report

No.	Title	Data Source
1	Dataset Information	Notebook Cells 1-2 (df.shape, df.isnull(), df.duplicated())
2	Class Distribution	Notebook Cell 2 (df["Result"].value_counts())
3	Experimental Setup	Notebook Cells 3, 4 (train_test_split, model definitions)
4	Software / Libraries Used	Notebook import statements, Cells 1-12
5	Model Performance Comparison	Notebook Cell 5 (results_df)
6	Confusion Matrix Results	Notebook Cell 6 (confusion_matrix per model)
7	Random Forest Feature Importance (Top 15)	Notebook Cell 8 (feature_importance dataframe)
8	Comparison with Related Work	Stage 1 literature review

APPENDIX C - GITHUB DOCUMENTATION CHECKLIST

The following items are recommended for upload to the project's GitHub repository to accompany this Stage 2 report:

- Source notebook: `Phishing_Website_Detection.ipynb` (with all cell outputs saved, as used for this report).
- Dataset information / dataset link: reference to the UCI Machine Learning Repository, Dataset 327 (<https://archive.ics.uci.edu/dataset/327/phishing>), plus the `dataset.csv` file or a note on how to obtain it, given licensing/size permits.
- `README.md` describing the project, dataset, how to run the notebook, and a summary of the five-model results table.
- `requirements.txt` listing the exact library versions used (pandas, scikit-learn, matplotlib, seaborn, shap), to help reproduce - and to help debug - the SHAP compatibility issue noted in Section 9.2.
- Results/figures folder containing the exported confusion-matrix, ROC-curve and feature-importance images used in this report.
- This Stage 2 project report (PDF/DOCX).
- Any additional source code files, if the pipeline is later refactored out of the notebook into standalone `.py` scripts.